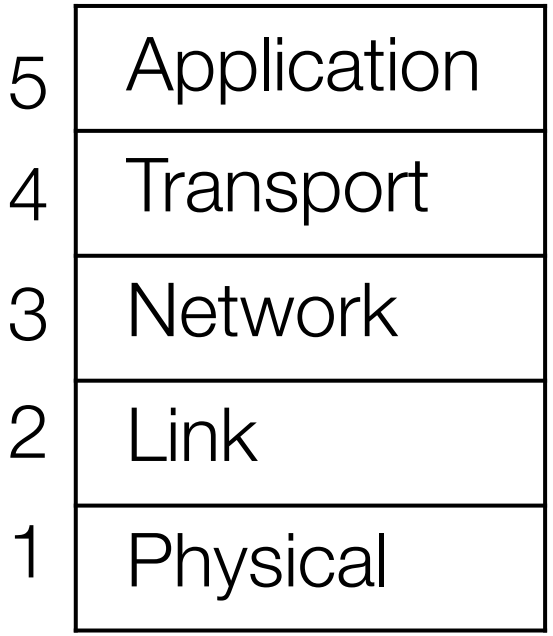
Network Programming
Application Layer

Minchen Yu
SDS@CUHK-SZ
Spring 2026

香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen

SCHOOL OF
DATA SCIENCE
數據科學學院

Network layers



Programs that use network service

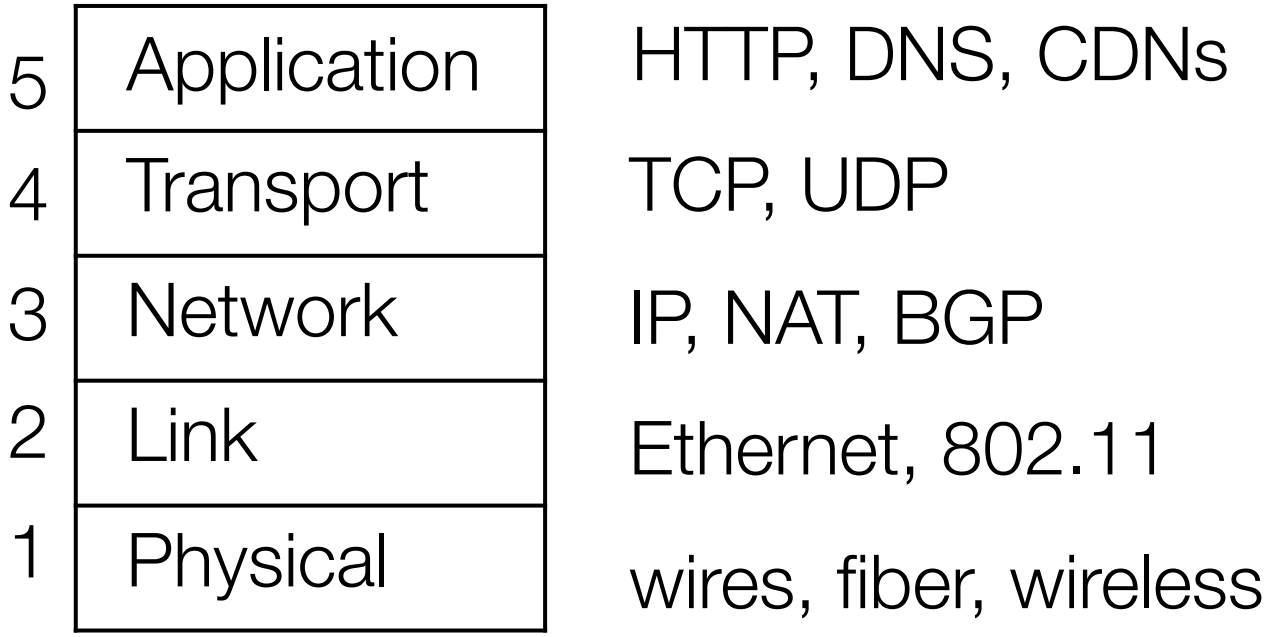
Provides end-to-end data delivery

Send packets over multiple networks

Send frames over one or more links

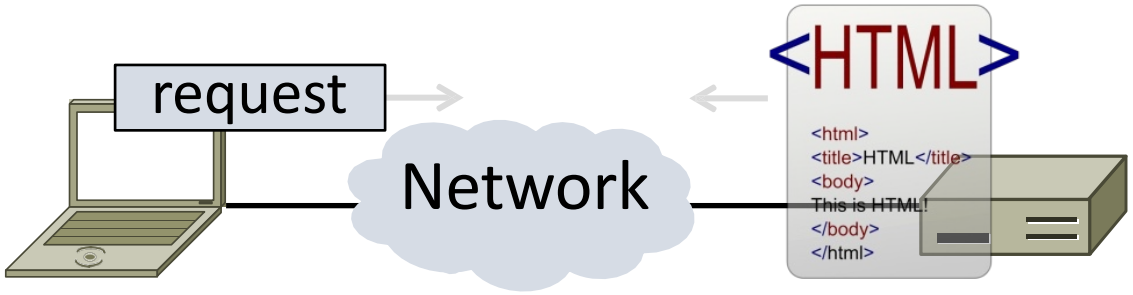
Send bits using signals

Network layers



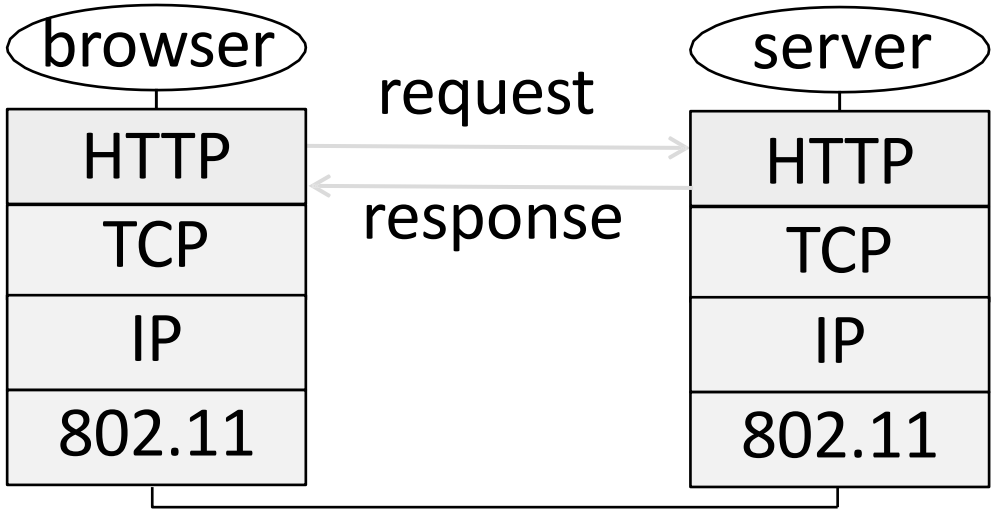
HTTP: HyperText Transfer Protocol

- Basis for fetching Web pages



Web Protocol Context

- HTTP is a request/response protocol
  - Runs on TCP, typically port 80
  - Part of browser/server app



Fetching a Web page with HTTP

- Start with the page URL (Uniform Resource Locator):

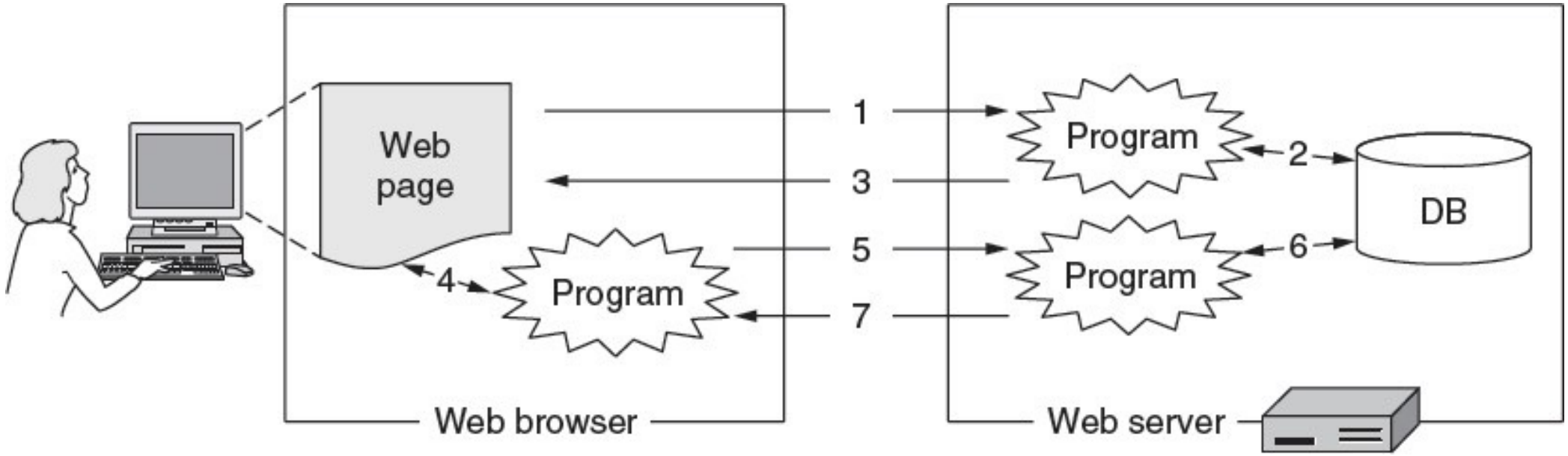
  http://en.wikipedia.org/wiki/Vegemite

  Protocol | Server | Page on server

- Steps:
  1. Resolve the server to IP address (DNS)
  2. Set up TCP connection to the server
  3. Send HTTP request for the page
  4. Await HTTP response for the page
  5. Execute and fetch embedded resources, render
  6. Clean up any idle TCP connections

Static vs Dynamic Web pages

- Static: Just static files, e.g., image
- Dynamic: Page content based on some computation
  - Javascript on client, PHP on server, or both



HTTP Protocol

- Originally simple; many options added over time
  - Text-based commands, headers

- Try it yourself: As a “browser” fetching a URL
  - Run “telnet <server name> 80”
  - Enter “GET /index.html HTTP/1.1”
  - Server will return HTTP response

HTTP Protocol

- Commands used in the request

Fetch page →

Upload data →

Method	Description
GET	Read a Web page
HEAD	Read a Web page's header
POST	Append to a Web page
PUT	Store a Web page
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Connect through a proxy
OPTIONS	Query options for a page

← Basically defunct

HTTP Protocol

- Codes returned with the response

Yes! →

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

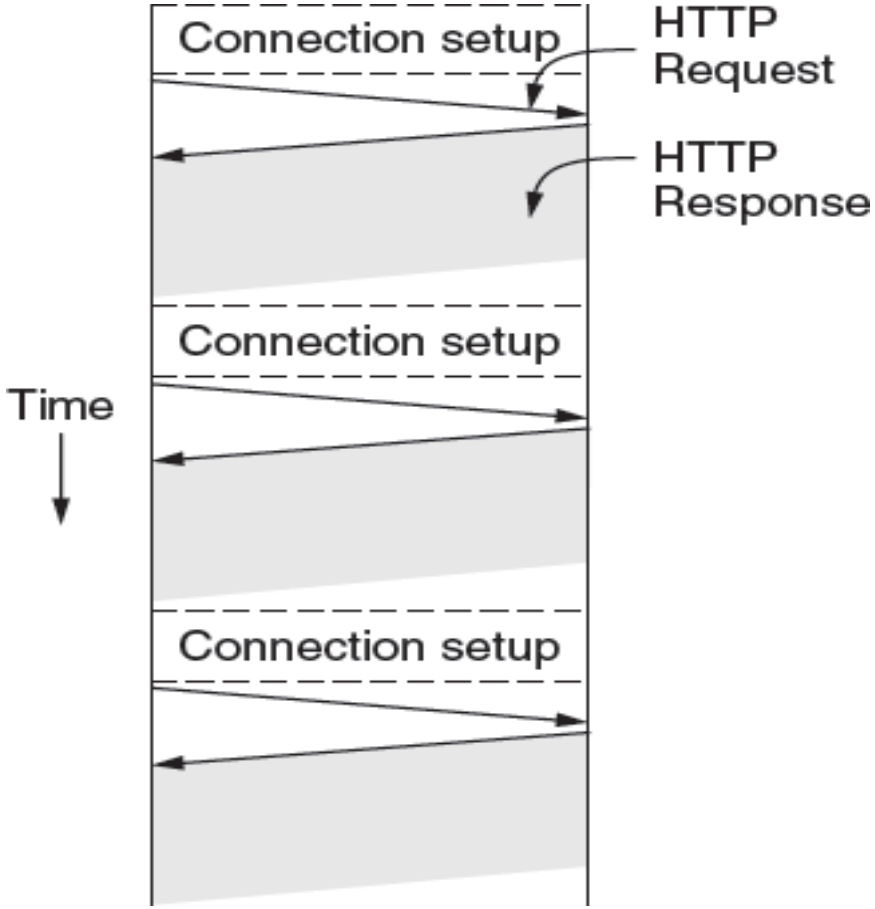
Performance

PLT (Page Load Time)

- PLT is a key measure of web performance
  - From click until user sees page
  - Small increases in PLT decrease sales
- PLT depends on many factors
  - Structure of page/content
  - HTTP (and TCP!) protocol
  - Network RTT and bandwidth

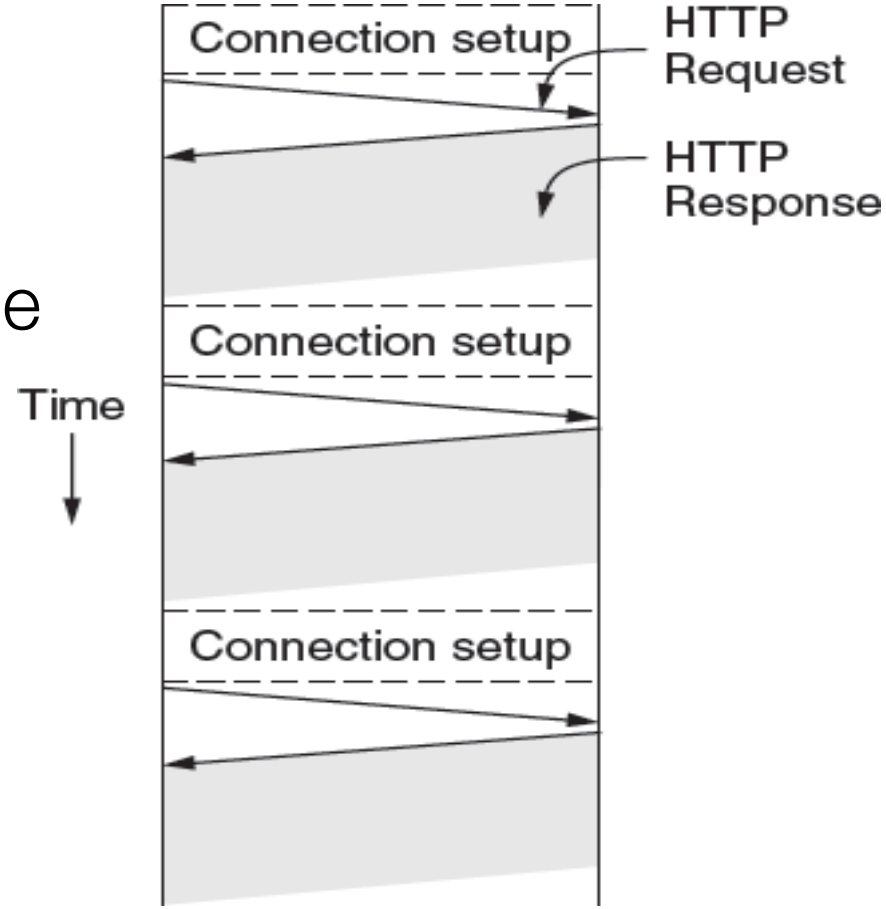
Early Performance

- HTTP/1.0 used one TCP connection per web resource
  - Made HTTP very easy to build
  - But gave fairly poor PLT…



Reasons for Poor PLT

- Sequential request/responses, even when to different servers
- Multiple TCP connection setups to the same server
  - Multiple TCP slow-start phases
- Network is not used effectively
  - Worse with many small resources



Ways to Improve PLT

- Reduce content size for transfer
  - Smaller images, gzip
- Make better use of the network
  - Next
- Avoid fetching same content
  - Caching and proxies [later]
- Move content closer to client
  - CDNs [later later]

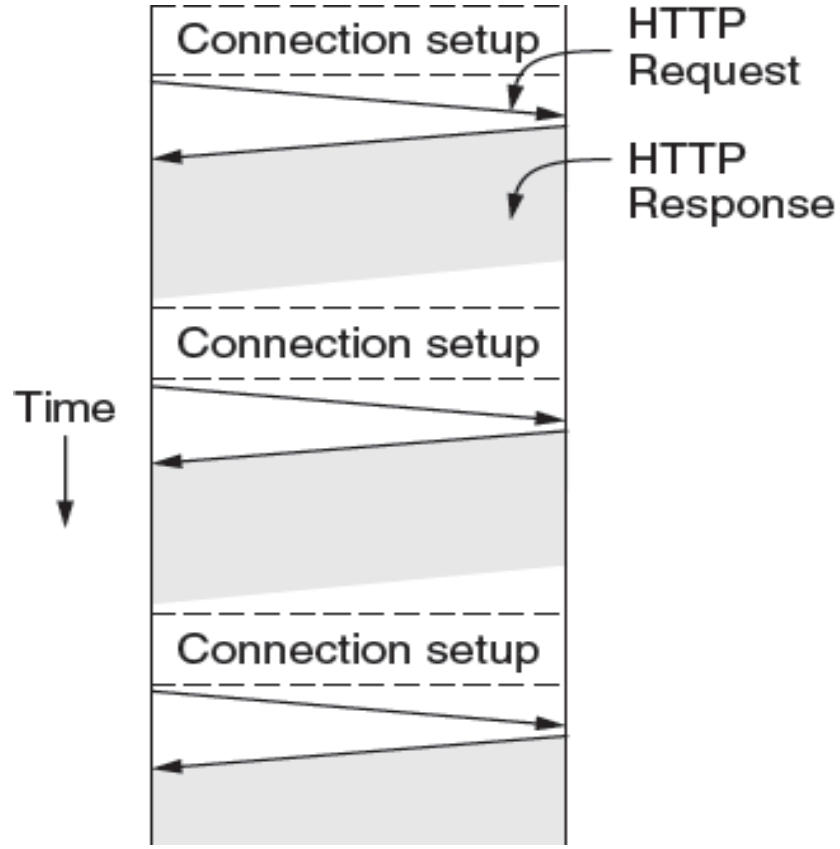
Better Network Use: Parallel Connections

- Browser runs multiple (say, 8) parallel HTTP instances
  - Server is unchanged; already handled concurrent requests for many clients
- How does this help?
  - Single HTTP wasn't using network much …
  - So parallel connections aren't slowed much
  - Pulls in completion time of last fetch

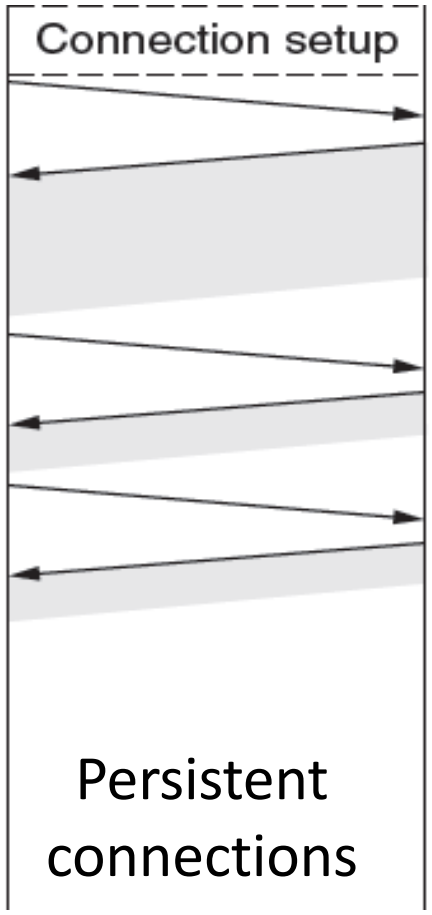
Better Network Use: Persistent Connections

- Parallel connections compete with each other for network resources
  - Exacerbates network bursts, and loss
- Persistent connections
  - Make 1 TCP connection to 1 server
  - Use it for multiple HTTP requests

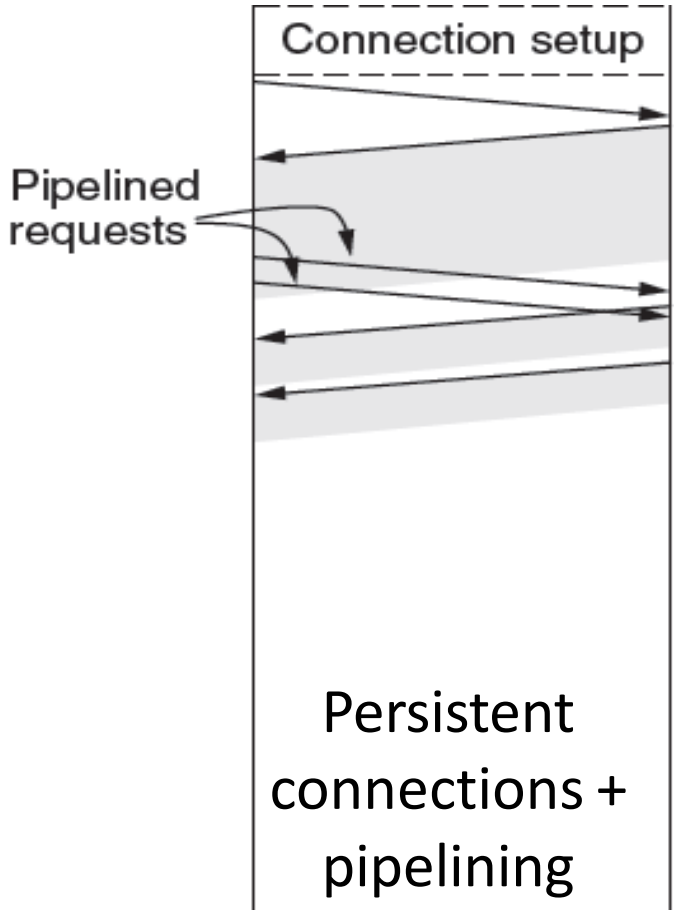
Persistent Connections



One request per connection



Persistent connections



Persistent connections + pipelining

Persistent Connections

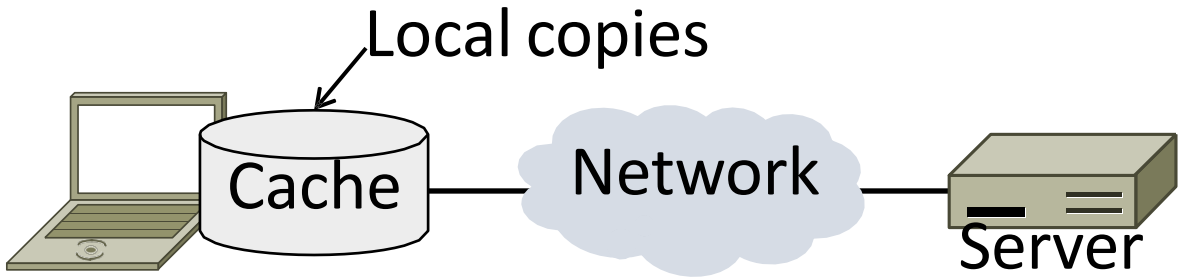
- Widely used as part of HTTP/1.1
  - Supports optional pipelining
  - PLT benefits depending on page structure, but easy on network

But we didn’t stop there ….

Web Caching and Proxies

Web Caching

- Users often revisit web pages
  - Big win from reusing local copy, aka, caching

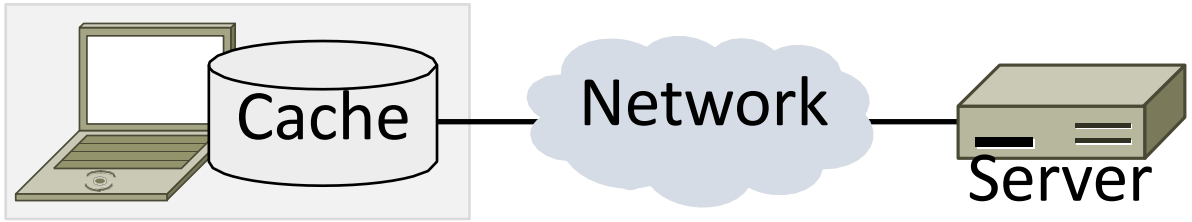


- Key question:

When is it OK to reuse local copy?

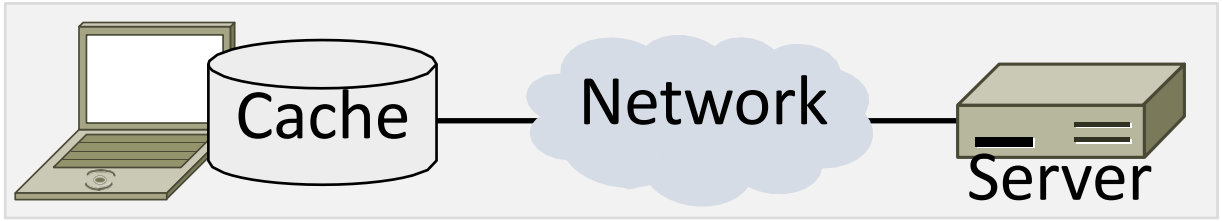
Locally Determine Validity of Cached Content

- Based on expiry information such as “Expires” header
- Or a heuristic (cacheable, fresh, not modified recently)
- Content is then available right away

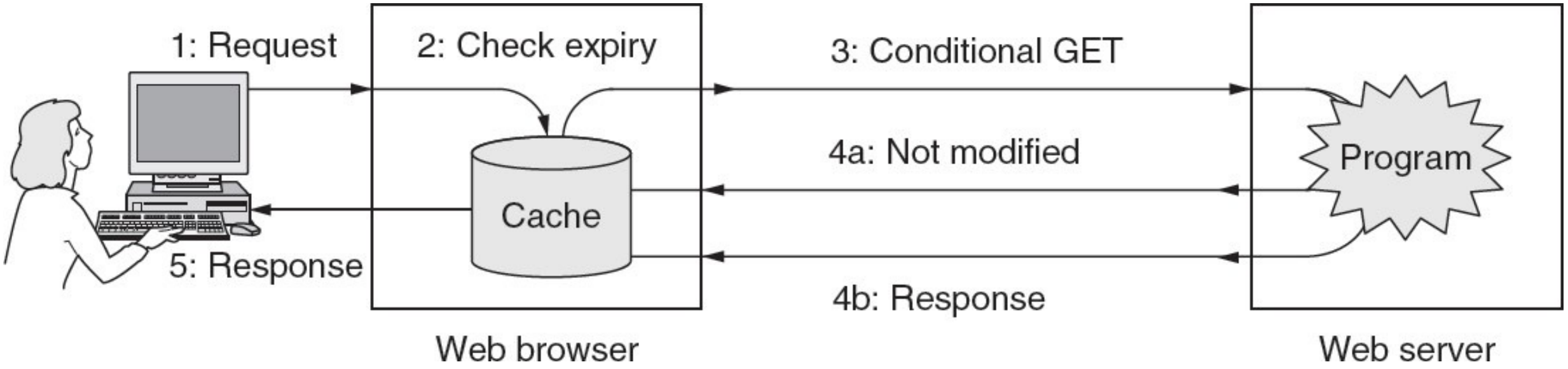


Use Server to Validate Cached Content

- Based on “Last-Modified” header from server
- Or based on “ETag” header from server
- Content is available after 1 RTT (if connection open)



Web Caching: Putting it together

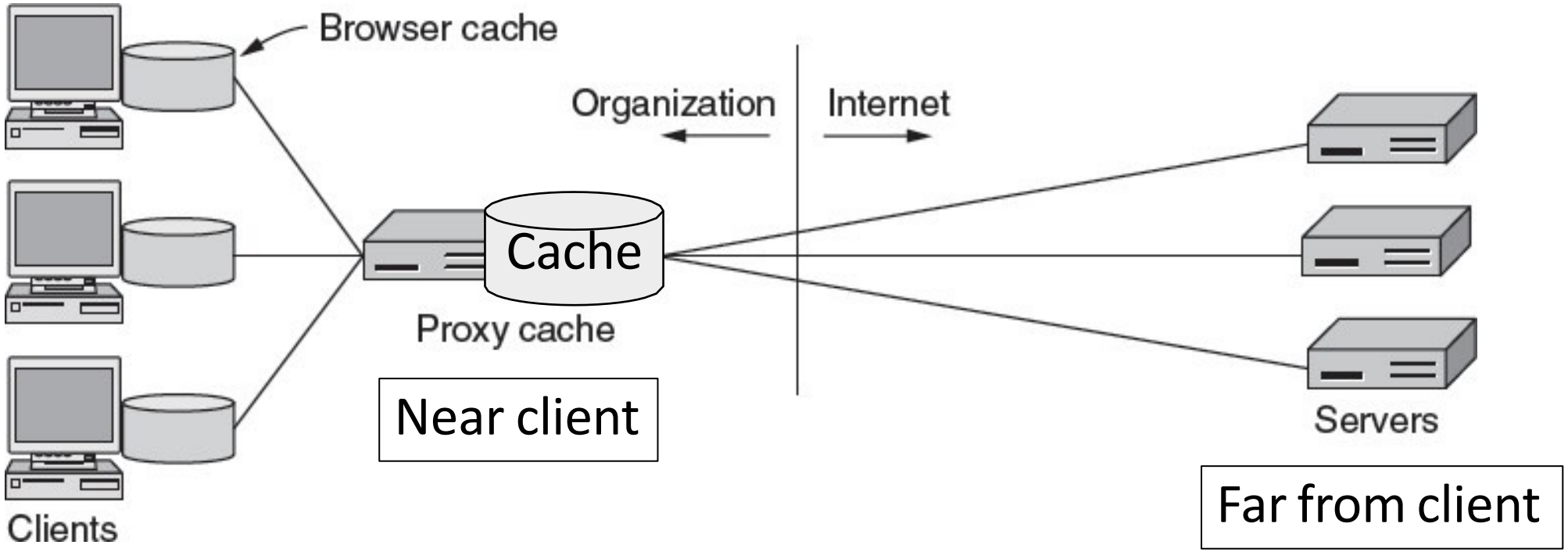


Web Proxies

- Place intermediary between clients and servers
- Benefits for clients include a shared cache
  - Limited by secure / dynamic content
  - Also limited by “long tail”
- Organizational access policies too!

Web Proxies in Action

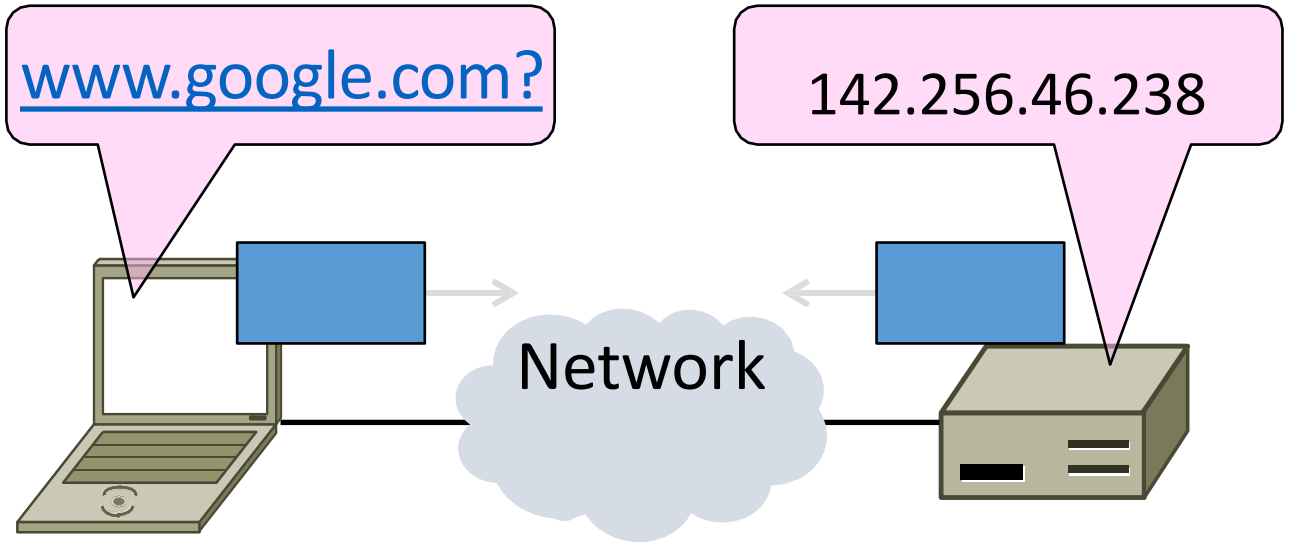
Clients contact proxy; proxy contacts server



Domain Name System (DNS)

DNS

- Human-readable host names, and more



Names and Addresses

- Names are higher-level identifiers for resources
- Addresses are lower-level locators for resources
  - Multiple levels, e.g. full name -> email -> IP address -> Ethernet addr
- Resolution (or lookup) is mapping a name to an address

Before the DNS – HOSTS.TXT

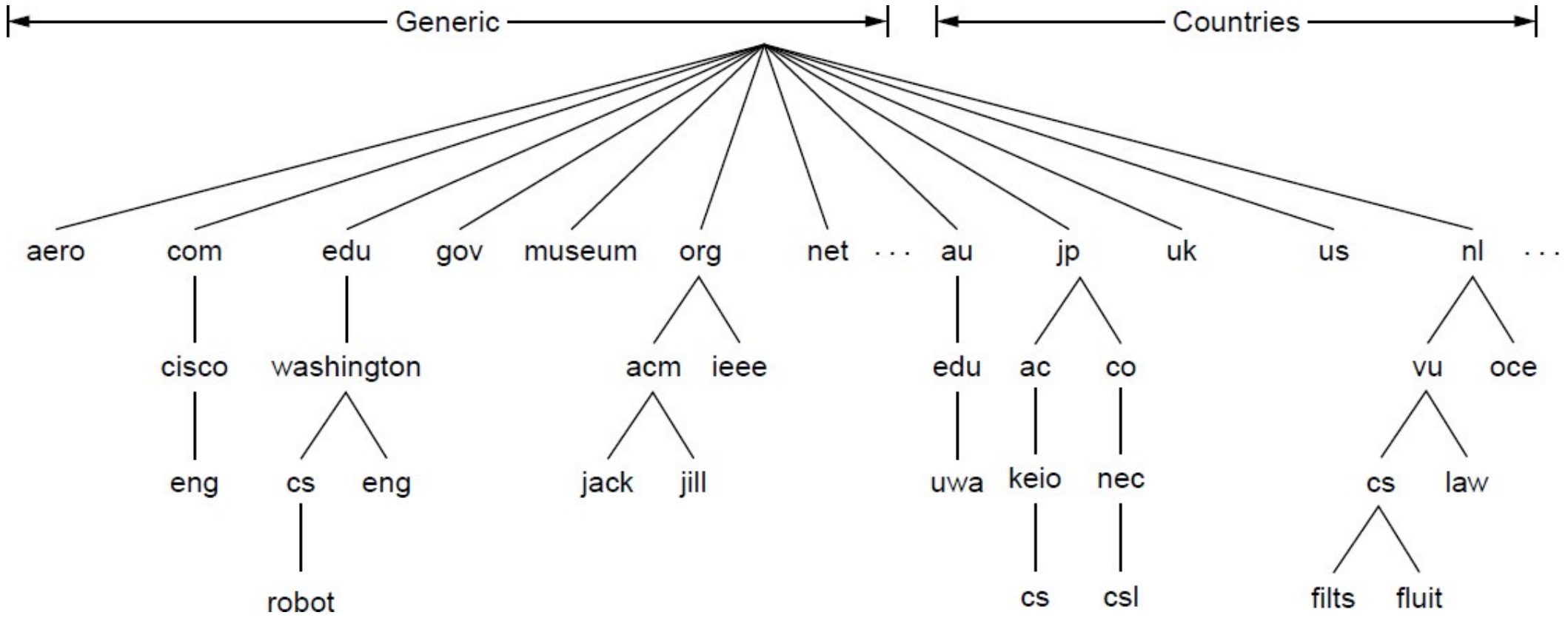
- Directory was a file HOSTS.TXT regularly retrieved for all hosts from a central machine at the NIC (Network Information Center)
- Names were initially flat, became hierarchical
- Not manageable or efficient as the ARPANET grew …

DNS

- A naming service to map between host names and their IP addresses (and more)
- Goals:
  - Easy to manage (esp. with multiple parties)
  - Efficient (good performance, few resources)
- Approach:
  - Distributed directory based on a hierarchical namespace
  - Automated protocol to tie pieces together

DNS Namespace

- Hierarchical, starting from "." (dot, typically omitted)



TLDs (Top-Level Domains)

- Run by ICANN (Internet Corp. for Assigned Names and Numbers)
  - Naming is financial, political, and international
- 700+ generic TLDs
  - Initially .com, .edu , .gov., .mil, .org, .net
  - Unrestricted (.com) vs Restricted (.edu)
  - Added regions (.asia, .kiwi), Brands (.apple), Sponsored (.aero) in 2012
- ~250 country code TLDs
  - Two letters, e.g., “.au”, plus international characters since 2010
  - Widely commercialized, e.g., .tv (Tuvalu)
  - Many domain hacks, e.g., instagr.am (Armenia)

DNS Zones

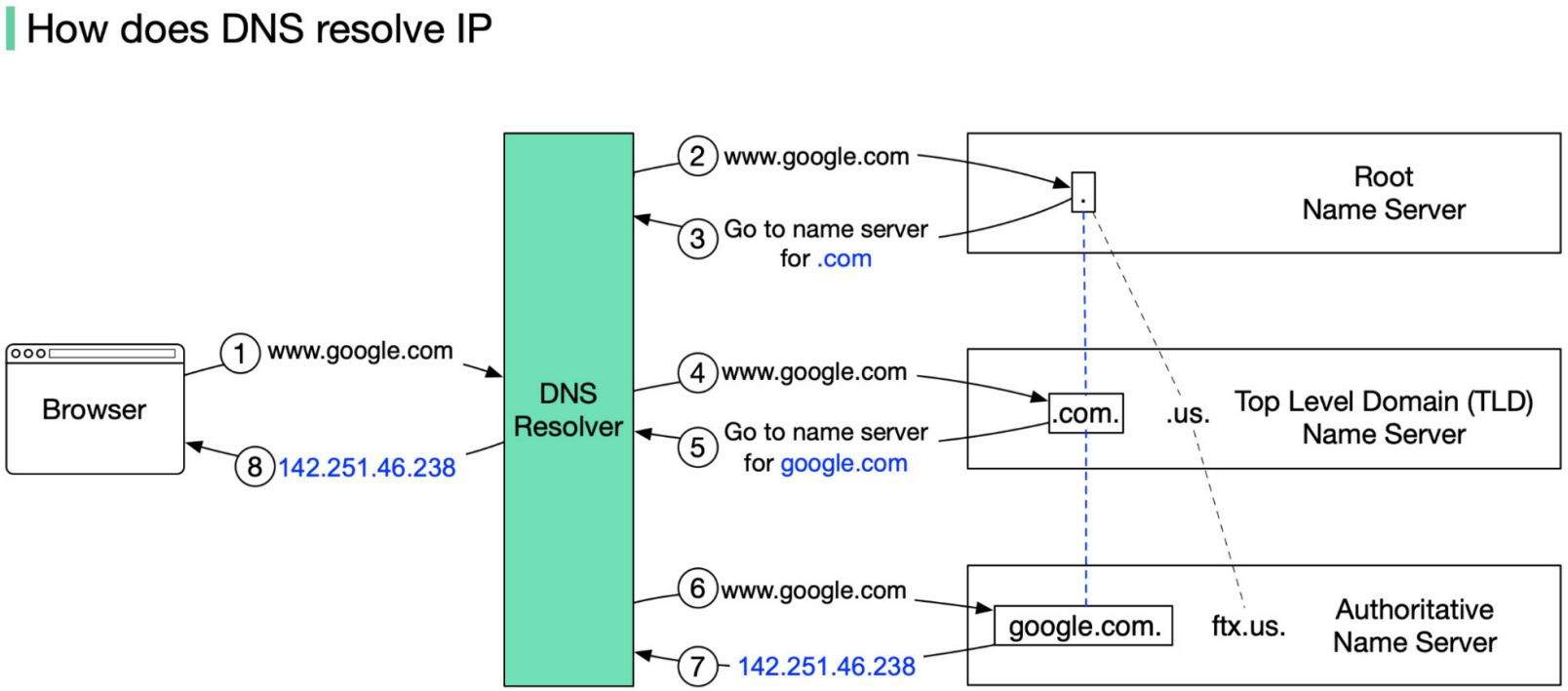
- Zones are the basis for distribution
  - Delegating the administration for a part of the DNS namespace
  - E.g., CUHK.EDU.CN created by EDU.CN administrators
- Each zone has a nameserver to contact for information about it
  - Zone must include contacts for delegations

DNS Resolution

- DNS protocol lets a host resolve any host name (domain) to IP address
- If unknown, can start with the root nameserver and work down zones
- Let’s see an example first …

DNS Resolution

- Example: resolve www.google.com



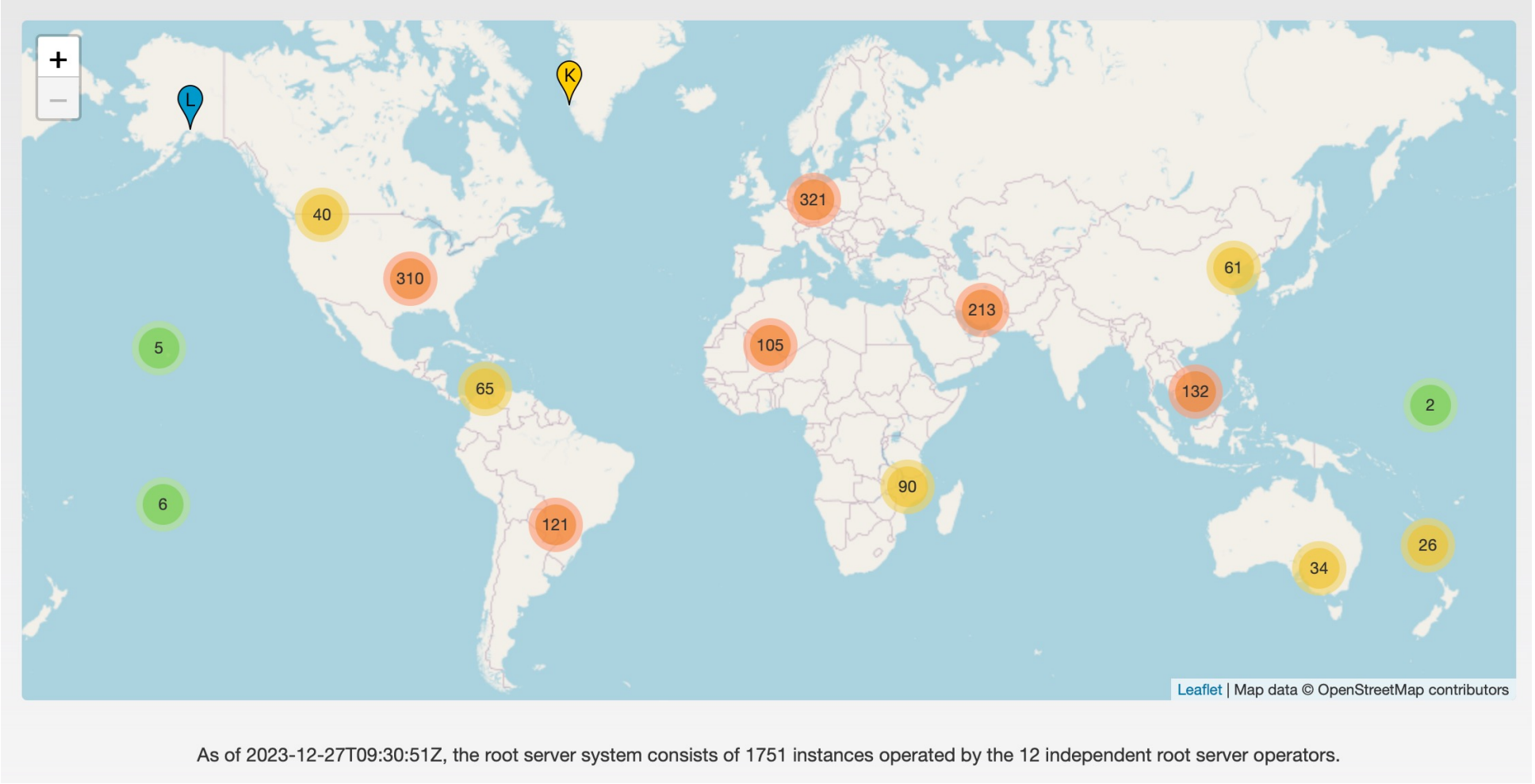
Local Nameservers

- Local nameservers often run by IT (enterprise, ISP)
  - But may be your host or AP
  - Or alternatives e.g., Google public DNS (8.8.8.8) Cloudflare's public DNS (1.1.1.1)
- Clients need to be able to contact local nameservers
  - Typically configured via DHCP

Root Nameservers

- Root (dot) is served by 13 server names
  - a.root-servers.net to m.root-servers.net
- Local name servers contact root servers when they cannot resolve a name
  - Configured with well-known root servers

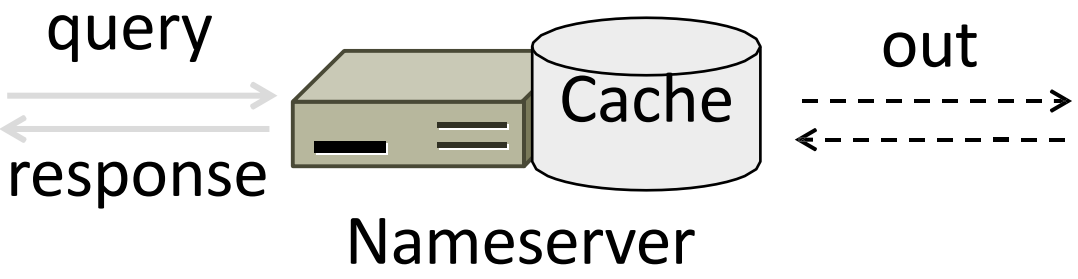
Root Server Deployment



Source: www.root-servers.org

Caching

- Goal: Low resolution latency
- Observation: Names don't have much churn
- Cache query/responses to answer future queries immediately
  - Responses carry a Time-To-Live(TTL) for caching



Caching

- Cached data periodically times out. How to choose TTL?
- Common practices
  - Top-level NS records: very high TTL
    - alleviate load on root
  - Intermediary NS records: high TTL
  - A records: small TTL (<7200s)
    - consistency concerns
  - Some A records: tiny TTL (<30s)
    - fault tolerance, load balancing

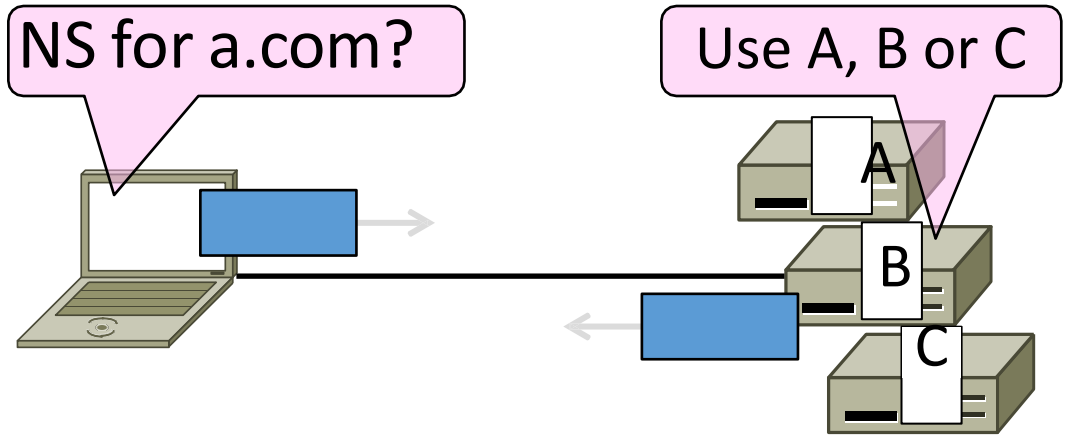
DNS Protocol

- Query and response messages
  - Built on UDP messages, port 53
  - Query using *dig*
  - E.g., dig www.google.com .Try it yourself

```
; <<>> DiG 9.10.6 <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 40832
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;www.google.com.                        IN      A

;; ANSWER SECTION:
www.google.com.         139     IN      A       216.58.200.228

;; Query time: 10 msec
;; SERVER: 10.20.232.47#53(10.20.232.47)
;; WHEN: Wed Dec 27 17:56:37 CST 2023
;; MSG SIZE  rcvd: 59
```

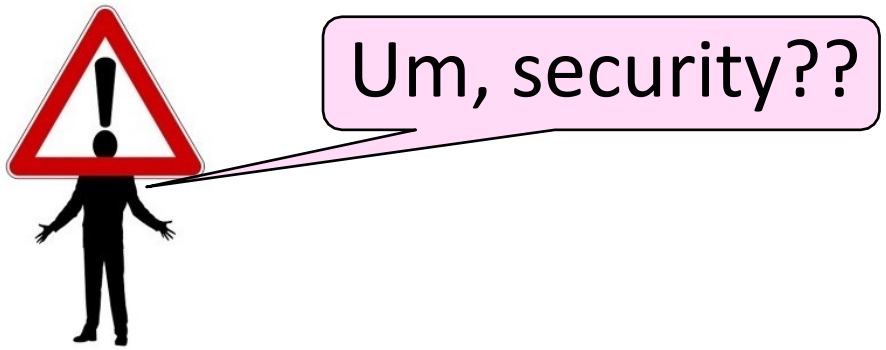
DNS Protocol

- Service reliability via replicas
  - Run multiple nameservers for domain
  - Return the list; clients use one answer
  - Helps distribute load too



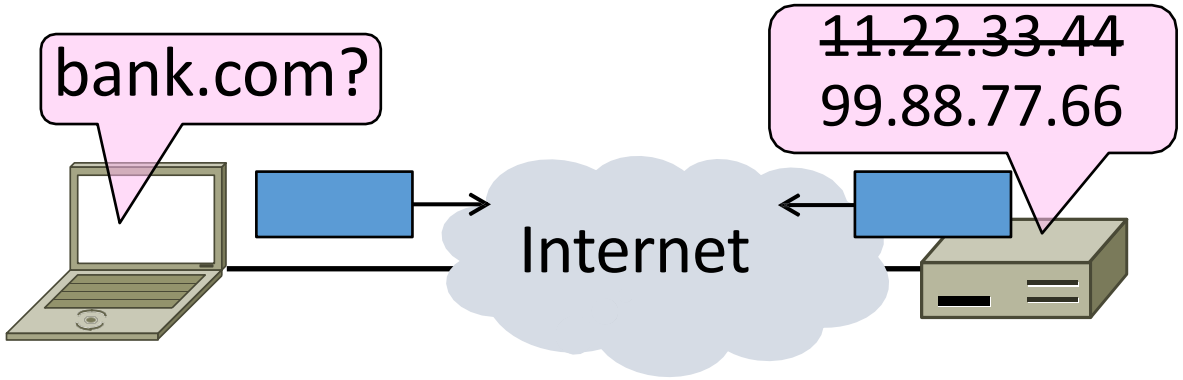
DNS Security

- Security is a major issue
  - Compromise redirects to wrong site!
  - Not part of initial protocols ..
- DNSSEC (DNS Security Extensions)
  - Mostly deployed



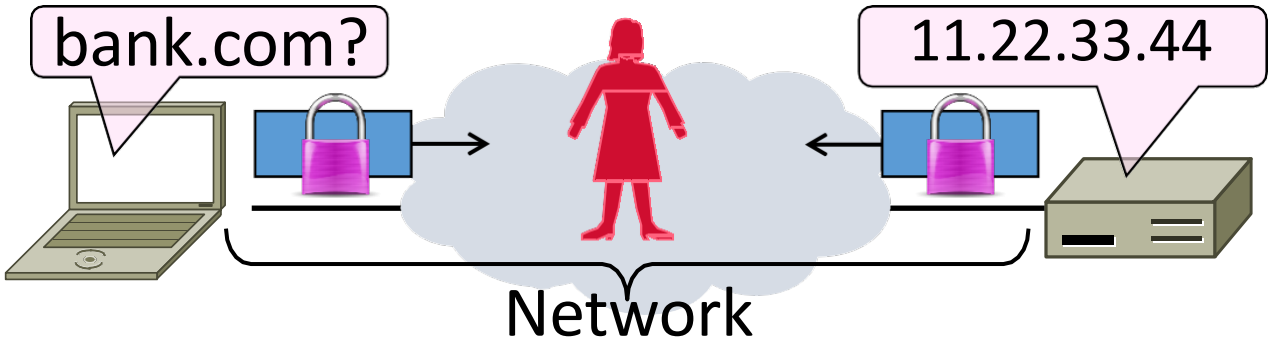
Goal and Threat Model

- Naming is a crucial Internet service
  - Binds host name to IP address
  - Wrong binding can be disastrous…



Goal and Threat Model

- Goal is to secure the DNS so that the returned binding is correct
  - Integrity vs confidentiality
- Attacker can tamper with messages on the network

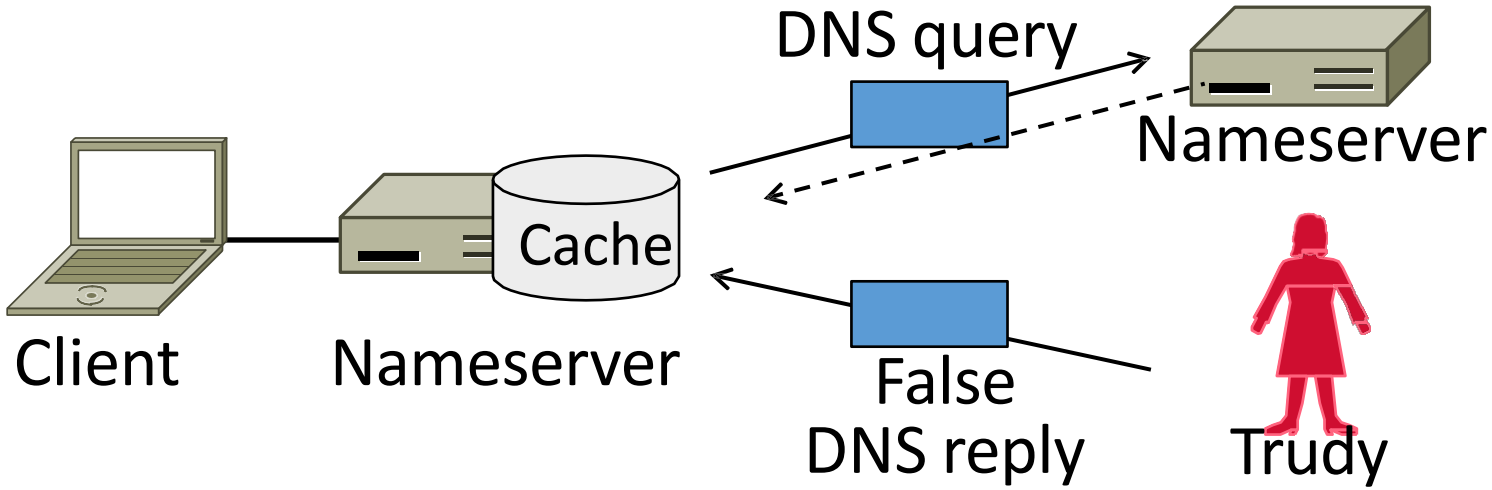


DNS Spoofing

- Attackers can trick nameserver into caching the wrong binding
  - By using the DNS protocol itself
  - This is called DNS spoofing

DNS Spoofing

- To spoof, Trudy returns a fake DNS response that appears to be true
  - Fake response contains bad binding



DNSSEC (DNS Security Extensions)

- Extends DNS with new record types
  - RRSIG for digital signatures of records
  - DNSKEY for public keys for validation
  - DS for public keys for delegation
- Deployment requires software upgrade at both client and server
  - Root servers upgraded in 2010
  - Followed by uptick in deployment

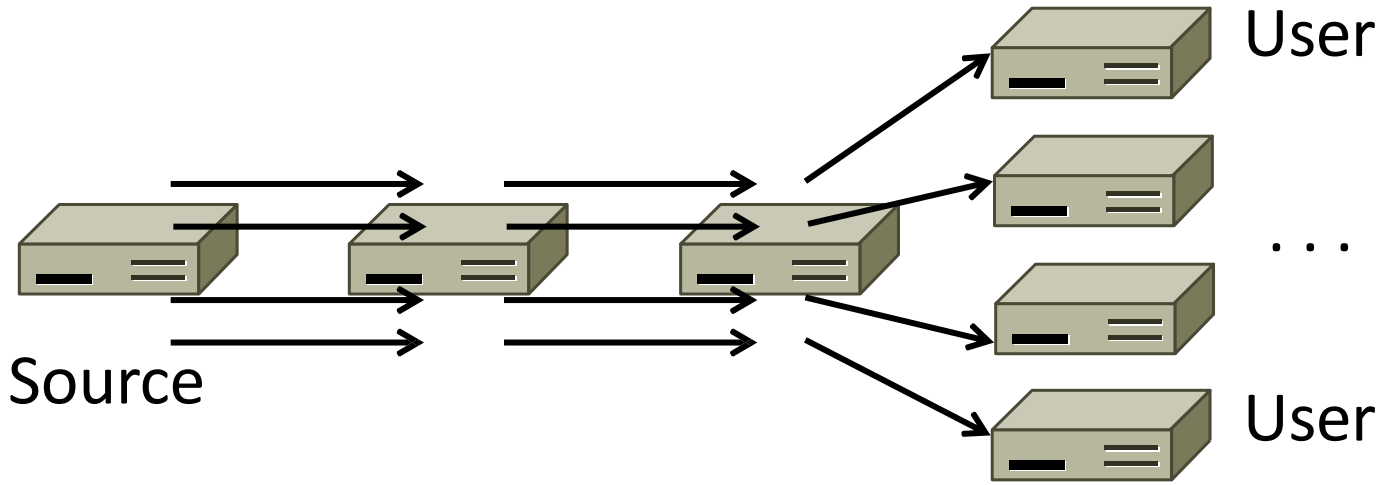
CDNs

Content Delivery Networks

- As the Web took off, traffic volumes grew and grew.
  - Concentrated load on popular servers
  - Led to congested networks
  - Gave a poor user experience
- Idea:
  - Place popular content near clients
  - Helps with all three issues above

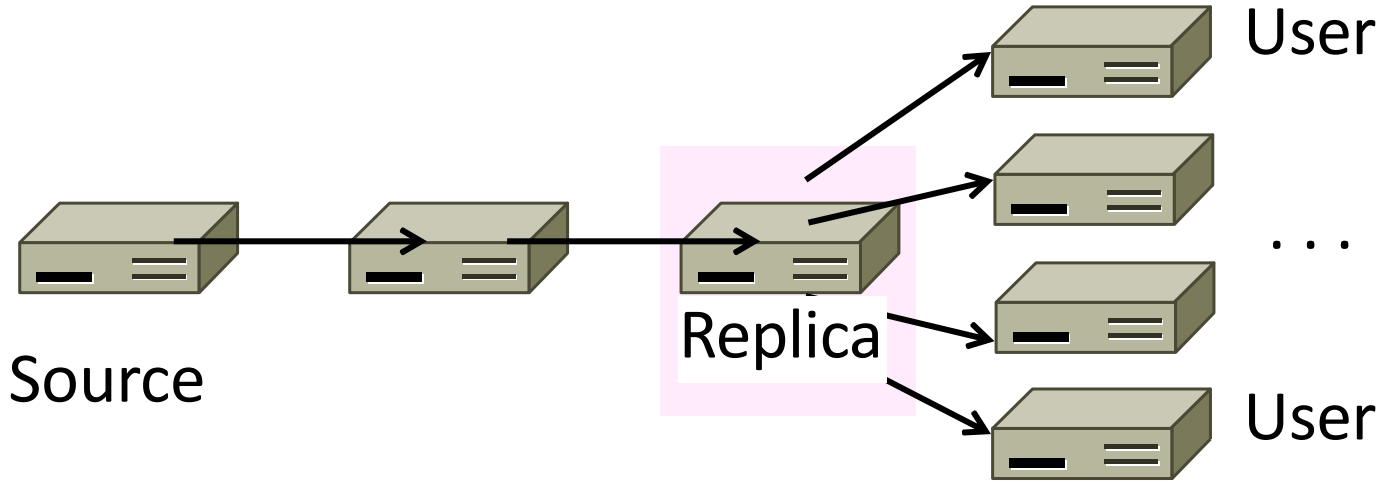
Before CDNs

- Sending content from the source server to 4 users takes 4 x 3 = 12 “network hops” in the example



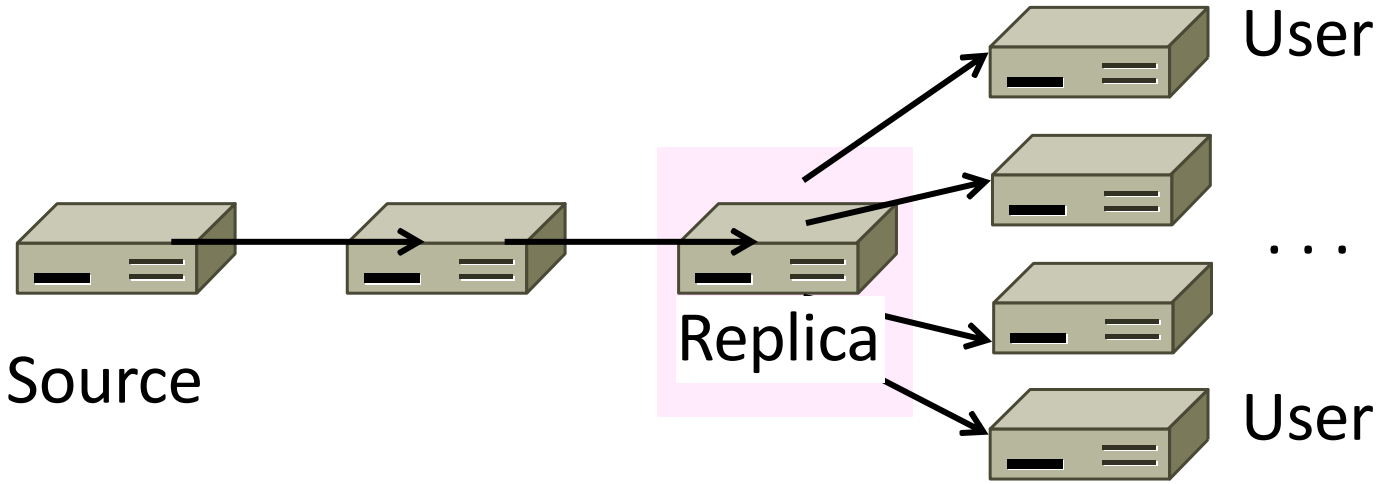
After CDNs

- Sending content via replicas takes only 4 + 2 = 6 “network hops”



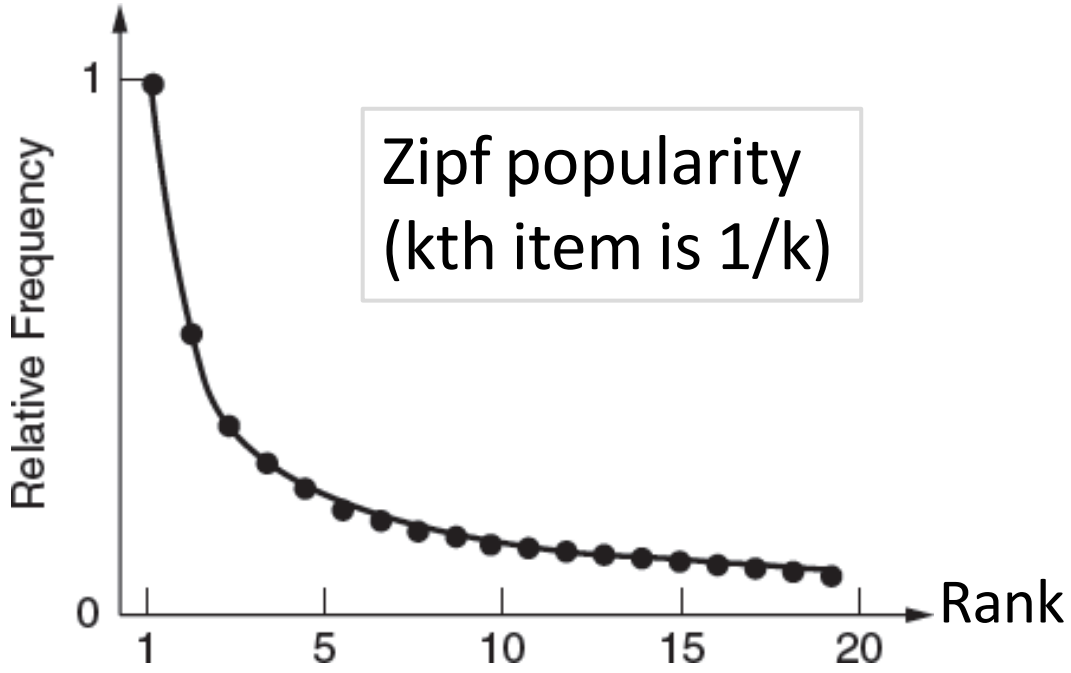
After CDNs

- Benefits assuming popular content:
  - Reduces source server, network load
  - Improves user experience



Popularity of Content

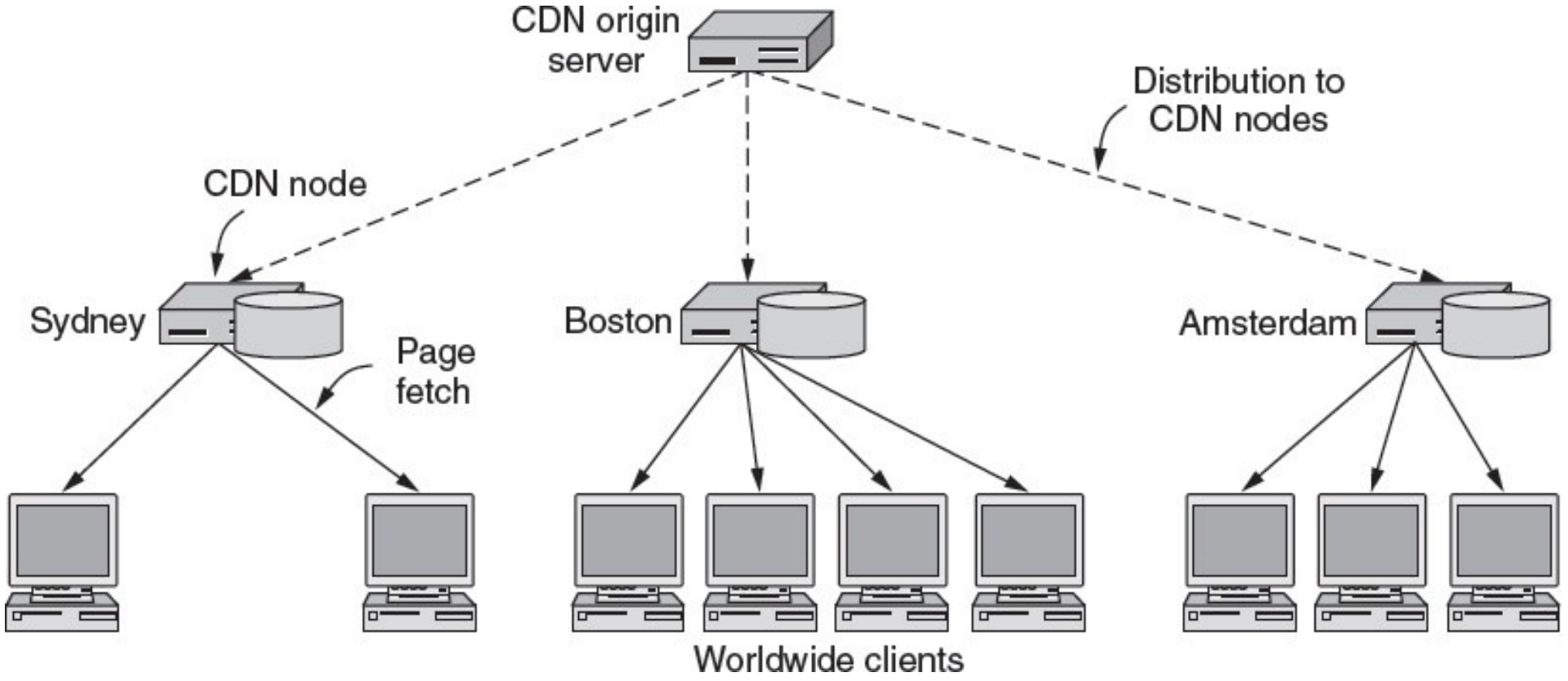
- Zipf's Law: few popular items, many unpopular ones; both matter



How to place content near clients?

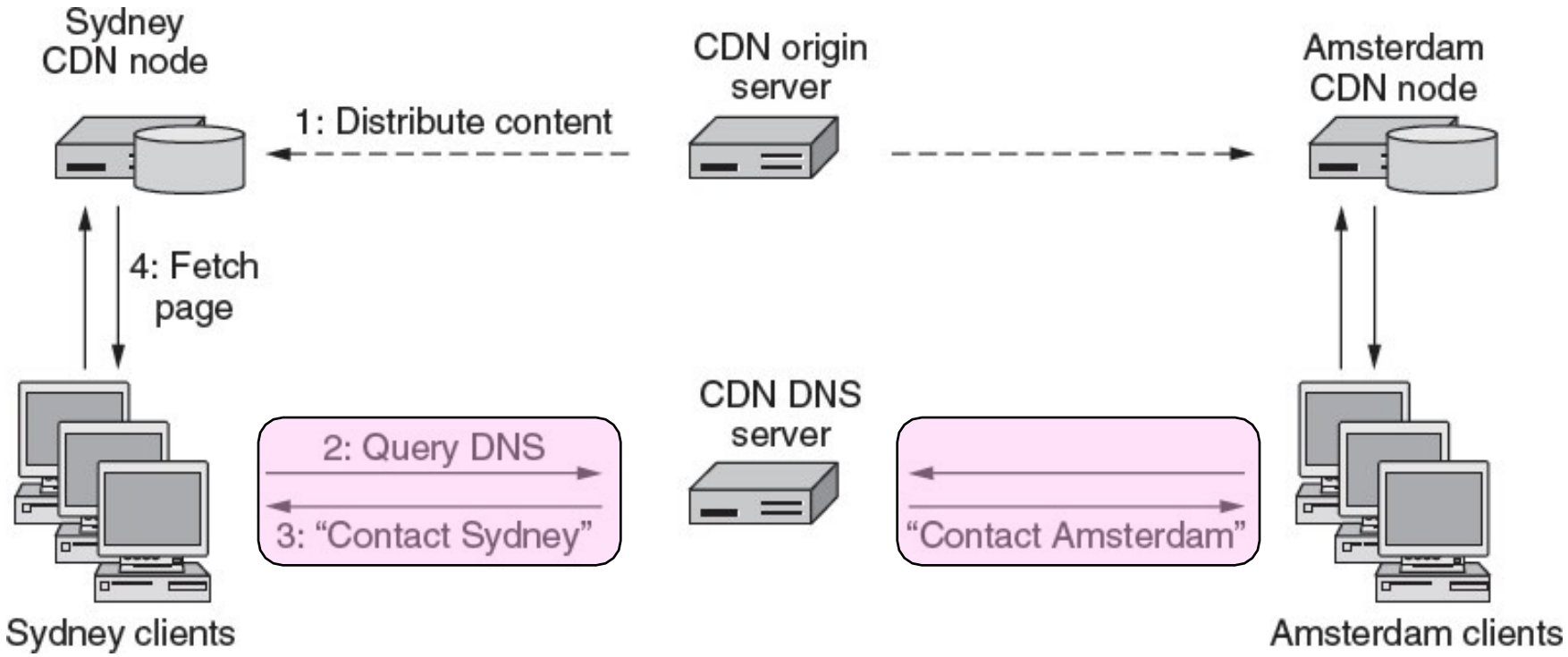
- Idea 1: Use browser and proxy caches
  - Helps, but limited to one client or clients in one organization
  - Want to place replicas across the Internet for use by all nearby clients
- Idea 2: Map clients to a nearby replica
  - Done via clever use of DNS

Content Delivery Network



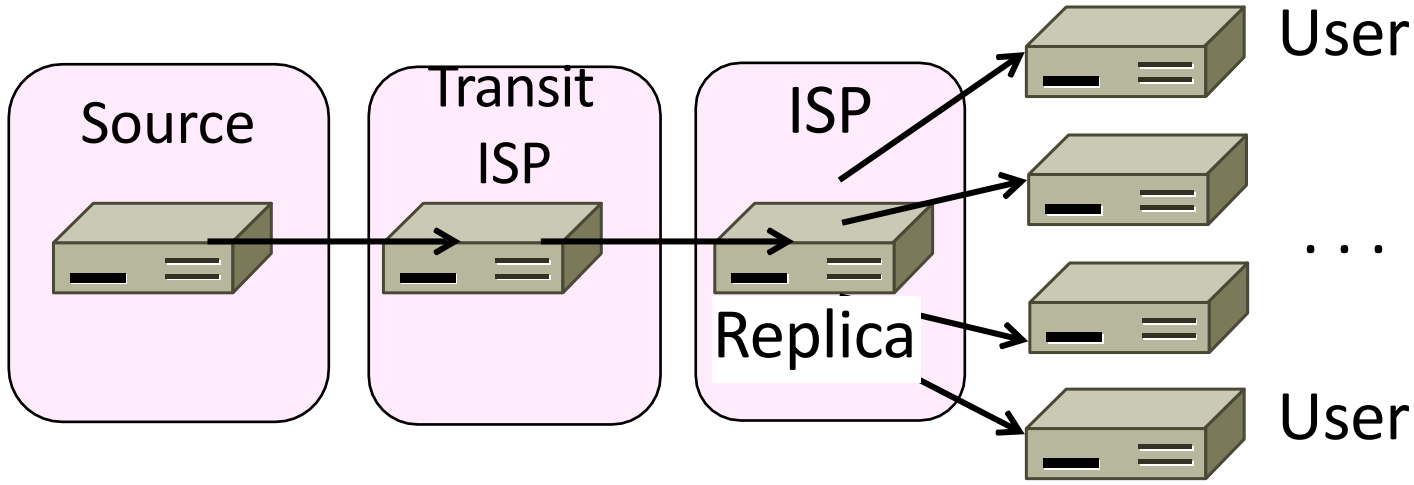
Content Delivery Network

- DNS gives different answers to clients
  - Tell each client the nearest replica (map client IP)



Business Model

- Placing site replica at an ISP is win-win
- Improves site experience and reduces ISP bandwidth usage



Credits

- Some slides are adapted from course slides of CSE 461 in UW